Actuariat des actifs et optimisation de portefeuille

Logique implémentée dans le simulateur de marché PIF : ce que fait chaque calcul, pourquoi, et comment il est codé

Rapport technique - 8 novembre 2026

> **À retenir.** La Bourse n'est pas assurable : le module « Actuariat » ne calcule donc **ni primes, ni sinistres**. Il applique les méthodes de mesure du risque des actuaires (volatilité, VaR, Expected Shortfall, drawdown, simulation) aux **actifs financiers**, et prévoit leur variation sous forme de **fourchettes de probabilités**, jamais comme une promesse de rendement. L'optimiseur de portefeuille (Markowitz) répond à une autre question : *comment répartir son capital entre plusieurs actifs* pour obtenir le meilleur compromis rendement / risque.

Contenu

- **1.** Vue d'ensemble et architecture du module Actuariat
- **2.** Les mesures de risque : volatilité, VaR, Expected Shortfall, drawdown
- **3.** La prévision de volatilité : EWMA et GARCH(1,1)
- **4.** La simulation Monte-Carlo
- **5.** Garde-fous, limites et tests
- **6.** L'optimisation de portefeuille : à quoi elle sert et comment elle est implémentée

Les chiffres des exemples proviennent du code réel du projet, exécuté sur des **données simulées** (aucun cours réel de la BVMT n'est utilisé ici). Ils servent à illustrer les méthodes, pas à dire quoi acheter.

1. Vue d'ensemble du module Actuariat

En assurance, l'actuaire estime des pertes futures à partir de statistiques du passé, puis fixe des réserves. Ici, on garde la même démarche, mais l'objet n'est plus un contrat : c'est un **actif coté**. La question devient : « à partir de l'historique de cet actif, quelle perte est plausible, et dans quelle fourchette son prix peut-il évoluer ? »

Le module répond avec deux familles de résultats :

- **Risque** (ce qui s'est passé) : volatilité, VaR, Expected Shortfall, drawdown maximal.
- **Prévision** (ce qui peut se passer) : volatilité attendue les prochains jours, puis simulation Monte-Carlo de milliers de trajectoires de prix.

Architecture : qui fait quoi

Couche	Fichier	Rôle
Écran (React)	ActuarialPage.jsx, actuarialService.js	Formulaire de paramètres, cartes de résultats et trois graphiques (drawdown, volatilité prévue, éventail Monte-Carlo). Onglet « Actuariat » de l'arène.
Backend (Spring)	ActuarialController, ActuarialService, ActuarialAiClient	Route POST /api/v1/actuarial/analyze protégée par JWT. Lit l'historique CSV local de l'actif, filtre les dates, envoie les clôtures au service IA.
Service IA (FastAPI)	ai/actuarial/api.py, risk.py, forecast.py	Route POST /api/ai/actuarial/analyze. Tous les calculs sont faits ici (numpy, scipy).
Tests	ai/actuarial/tests, ActuarialControllerIntegrationTest	12 tests pytest sur les calculs et l'API, 1 test d'intégration Spring. Lancés par la CI.

Choix de conception important. Le service Python ne lit aucune donnée : c'est Spring qui lui envoie les dates et les clôtures. Deux avantages : un seul endroit connaît les fichiers de cours, et Spring peut **couper l'historique à la date simulée de la salle**. Sans cela, une analyse faite en novembre 2022 utiliserait des cours de décembre 2022 : c'est une fuite d'information qui rendrait les résultats faux pour un joueur.

2. Les mesures de risque

Toutes les mesures partent des **rendements logarithmiques journaliers**, et toutes les pertes sont exprimées en nombres positifs (0,05 = perte de 5 %).

2.1 Rendement logarithmique

$$r_t = ln(P_t / P_{t-1})$$

Pourquoi le logarithme ? Les rendements log s'additionnent dans le temps (le rendement sur 10 jours est la somme des 10 rendements journaliers), ce qui simplifie les calculs de volatilité et de simulation. Le code refuse toute clôture nulle, négative ou non finie.

2.2 Volatilité annualisée

$$\sigma_{annuelle} = \sigma_{jour} \times \sqrt{252}$$

σ_{jour} est l'écart-type des rendements journaliers. On multiplie par √252 car il y a environ 252 séances par an, et que la variance croît proportionnellement au temps. **Exemple.** Sur la série simulée (1 500 séances), la volatilité annualisée est de **19,84 %** : un actif qui varie typiquement de ± 20 % sur un an.

2.3 Value at Risk (VaR)

La VaR à 95 % répond à : « quelle perte ne sera dépassée que 5 % des jours ? ». Le code en calcule deux versions :

- **VaR historique** : on prend le quantile à 5 % des rendements observés. Aucune hypothèse sur la forme de la distribution. VaR = $-q_{5\,\%}(r)$.
- **VaR paramétrique** : on suppose des rendements normaux, VaR = $-(\mu + z \times \sigma)$ avec $z = \Phi^{-1}(5\,\%) \approx -1{,}645$.

Exemple chiffré. VaR historique à 95 % sur 1 jour : **2 03 %**. Pour un portefeuille de 100 000 TND investi dans cet actif cela signifie : « 95 jours sur 100 on ne perd pas plus de 2 030 TND en une séance ». La VaR paramétrique donne 2,03 % : les deux sont proches parce que cette série est presque normale (kurtosis en excès : 0,86). Sur un actif réel aux queues épaisses, la VaR historique est en général la plus prudente.

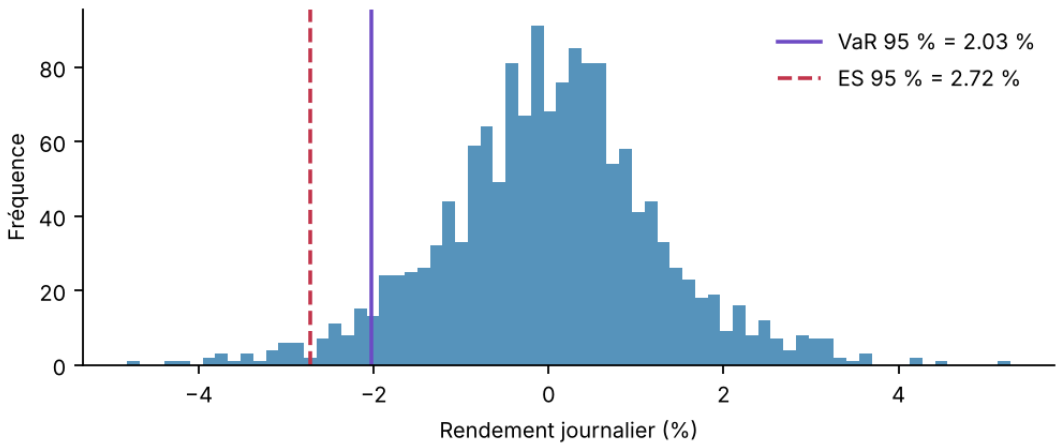


Figure 1 - Distribution des rendements journaliers (série simulée) avec la VaR et l'Expected Shortfall à 95 %.

2.4 Expected Shortfall (ES)

La VaR dit « jusqu'où on perd dans 95 % des cas », mais ne dit rien sur les 5 % restants. L'Expected Shortfall comble ce trou : c'est la **perte moyenne les jours où la VaR est dépassée**.

$$ES = -\text{moyenne des rendements} \leq \text{quantile à 5 \%}$$

Ici : ES à 95 % = **2,72 %** contre une VaR de 2,03 %. L'ES est toujours supérieure ou égale à la VaR (c'est vérifié par un test). À 99 %, la VaR monte à 3,12 % et l'ES à 3,72 % : plus on exige de confiance, plus la perte « possible » augmente.

2.5 Passage à un horizon de h jours

$$VaR_{h\ jours} = VaR_{1\ jour} \times \sqrt{h}$$

C'est la règle de la racine du temps, valable si les rendements sont indépendants d'un jour à l'autre. Exemple pour h = 10 : 2,03 % × √10 = **6,42 %** ; ES sur 10 jours = 8,61 %. C'est une approximation : en période de crise, les pertes sont corrélées d'un jour à l'autre et la règle sous-estime le risque.

2.6 Drawdown maximal

Le drawdown mesure la chute depuis le dernier sommet : $DD_t = 1 - P_t / \max_{s \leq t}(P_s)$. Le **drawdown maximal** est la pire chute sommet-creux de l'historique. C'est la mesure la plus parlante pour un joueur : « si j'avais acheté au pire moment, combien aurais-je vu fondre mon capital avant de remonter ? »

Exemple. Sur la série simulée : drawdown maximal = **26,66 %** (du sommet à la séance 903 au creux à la séance 1419) ; drawdown actuel = 11,64 %. Le code indique aussi si l'actif a retrouvé son sommet (« recovered »).

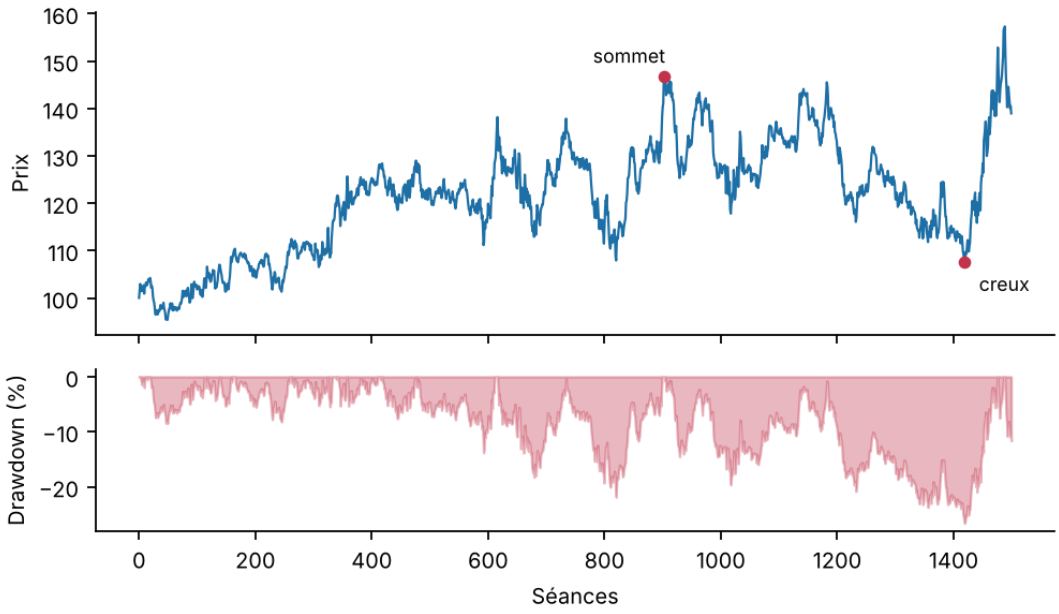


Figure 2 - Prix (haut) et drawdown (bas). Les points rouges marquent le sommet et le creux de la pire chute.

2.7 Asymétrie et queues épaisses

Le module calcule aussi le **skewness** (asymétrie : une valeur négative signale des pertes plus brutales que les gains) et le **kurtosis en excès** (0 pour une loi normale ; plus il est élevé, plus les événements extrêmes sont fréquents). Un kurtosis élevé est un avertissement : la VaR paramétrique et le Monte-Carlo gaussien sous-estiment alors le risque, et il vaut mieux choisir la loi de Student dans l'écran.

3. La prévision de volatilité

La volatilité n'est pas constante : les périodes calmes et les périodes agitées se regroupent (« clusters »). Prévoir la volatilité des prochains jours est donc plus fiable que prévoir le rendement, et c'est ce que fait le module. Deux modèles sont disponibles.

3.1 EWMA (RiskMetrics)

$$\sigma^2_t = \lambda \cdot \sigma^2_{t-1} + (1 - \lambda) \cdot r^2_{t-1} \text{ avec } \lambda = 0{,}94$$

La variance du jour est une moyenne pondérée de la variance de la veille (94 %) et du carré du dernier rendement (6 %) : les chocs récents comptent plus que les anciens. Le modèle donne une volatilité **constante** sur tout l'horizon de prévision. Il sert aussi de **solution de repli** quand l'historique est trop court pour GARCH.

3.2 GARCH(1,1)

$$\sigma^2_t = \omega + \alpha \cdot r^2_{t-1} + \beta \cdot \sigma^2_{t-1}$$

Même idée, mais avec trois paramètres estimés sur les données : ω (niveau de fond), α (réaction au dernier choc) et β (mémoire de la volatilité passée). L'estimation se fait par **maximum de vraisemblance** (scipy, algorithme SLSQP) sur les rendements centrés et exprimés en %, avec les contraintes $\alpha, \beta \geq 0$ et $\alpha + \beta < 0{,}999$ pour garder un modèle stable. Si l'ajustement échoue, si $\alpha + \beta$ est trop proche de 1, ou s'il y a moins de 100 observations, le code bascule automatiquement sur l'EWMA.

Pour prévoir k jours à l'avance, la variance revient progressivement vers sa moyenne de long terme $V = \omega / (1 - \alpha - \beta)$:

$$E[\sigma^2_{t+k}] = V + (\alpha + \beta)^k \cdot (\sigma^2_{t+1} - V)$$

Exemple. Sur la série simulée (qui se termine par une période agitée), le modèle estime α = 0,124, β = 0,848, soit une persistance α + β = 0,972 : un choc de volatilité met longtemps à s'estomper. La volatilité annualisée prévue démarre à **25,5 %** et redescend vers **21,3 %** en 60 jours, pour une moyenne de long terme de 20,3 %.

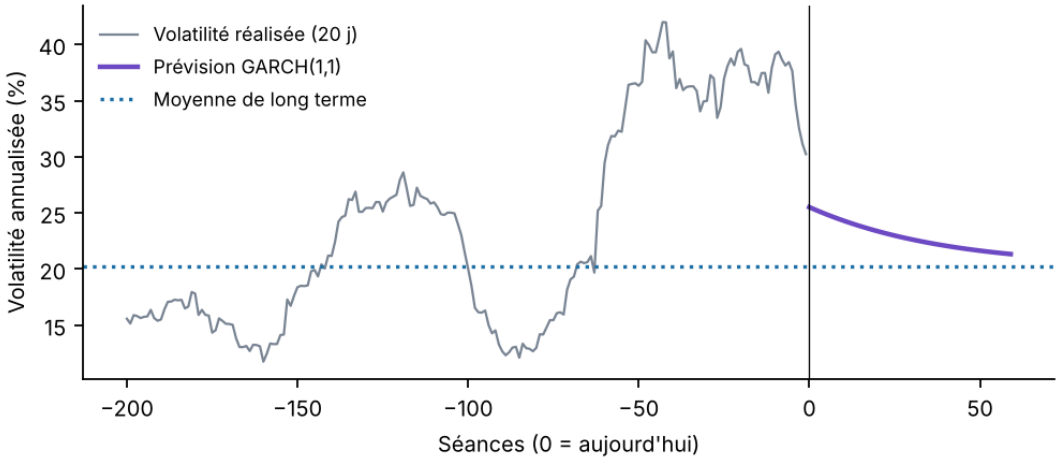


Figure 3 - Volatilité réalisée (gris) et prévision GARCH (violet) qui revient vers sa moyenne de long terme (pointillé).

4. La simulation Monte-Carlo

Une fois la volatilité des prochains jours prévue, on simule des milliers de trajectoires de prix possibles pour en déduire une **fourchette**, pas une valeur unique.

$$ln(P_{t+1} / P_t) = (dérive - \sigma^2_t / 2) + \sigma_t \cdot \varepsilon_t$$

- $\boldsymbol{\sigma_t}$: la volatilité journalière prévue à l'étape précédente (GARCH ou EWMA), jour par jour.
- $\boldsymbol{\varepsilon_t}$: le choc aléatoire. Au choix : loi **normale**, ou loi de **Student à 5 degrés de liberté** renormalisée pour avoir une variance de 1 : même volatilité, mais des événements extrêmes plus fréquents.
- **Dérive** : par défaut **nulle** (option « neutre »). La moyenne historique des rendements est très bruitée ; la projeter dans le futur serait une promesse que les données ne justifient pas. L'option « historique » existe pour comparer.
- $\boldsymbol{-\sigma^2/2}$: correction qui garantit que le prix moyen n'est pas faussé par l'effet du logarithme.

Le code génère 500 à 20 000 trajectoires (5 000 par défaut), puis en extrait : l'éventail de prix (percentiles 5, 25, 50, 75 et 95 % pour chaque jour), la probabilité de perte, la VaR et l'ES sur le prix final, et le drawdown moyen attendu sur la période. Une graine (seed) optionnelle rend la simulation reproductible, ce que les tests utilisent.

Exemple (actif simulé à 139,0, 30 jours, 5 000 trajectoires) :

Résultat	Valeur	Lecture
Prix médian	138,2	Proche du prix actuel : la dérive est nulle, on ne prétend pas prévoir une hausse.
Fourchette 90 %	121,0 - 158,2	9 trajectoires sur 10 finissent dans cet intervalle.
Probabilité de perte	52,6 %	Chance que le prix final soit sous le prix actuel (un peu plus de 50 % à cause de la correction $-\sigma^2/2$).
VaR 95 % à 30 jours	12,90 %	Perte dépassée dans 5 % des trajectoires.
Expected Shortfall 95 %	15,76 %	Perte moyenne dans ces 5 % de trajectoires.
Drawdown moyen attendu	8,4 %	Chute moyenne sommet-creux rencontrée pendant les 30 jours.

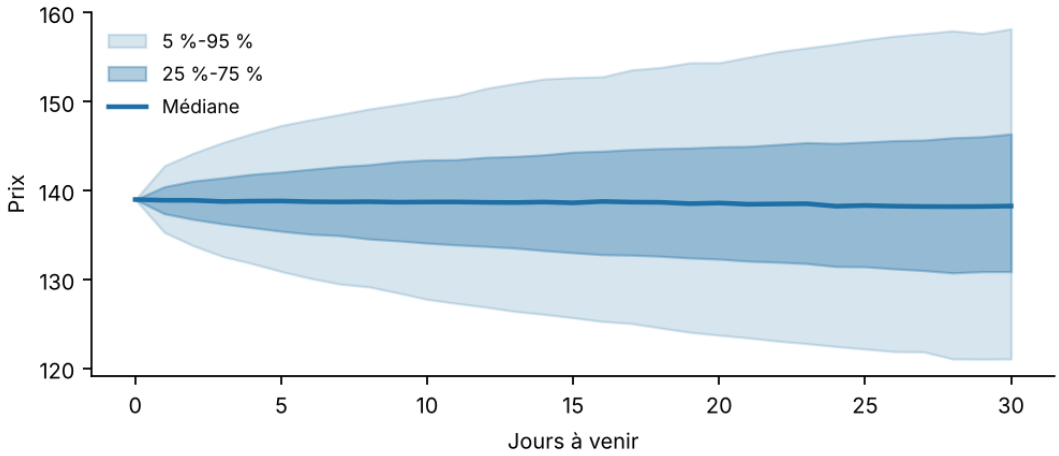


Figure 4 - Éventail Monte-Carlo : médiane (trait), bande 25-75 % (foncé) et bande 5-95 % (clair).

5. Garde-fous, limites et tests

Contrôles de validité

Paramètre	Bornes	Pourquoi
Historique minimum	60 séances	En dessous, volatilité et queues sont trop instables. Refus par Spring et par FastAPI (erreur 400 / 422).
Niveau de confiance	90 % à 99,9 %	Hors de cette plage, le quantile n'a pas de sens statistique.
Horizon de VaR / prévision	1 à 60 j / 1 à 90 j	Au-delà, la règle de la racine du temps et la prévision de volatilité ne sont plus fiables.
Simulations	500 à 20 000	Trop peu : résultat bruité. Trop : temps de calcul inutile.
Clôtures	strictement positives	Le logarithme est indéfini sinon. Dates et clôtures doivent avoir la même longueur.
Date de fin	date simulée de la salle	Aucune donnée postérieure n'est utilisée.

Limites à connaître

- **Le passé ne garantit rien.** Tout est calculé sur l'historique : un type de crise jamais observé n'apparaît pas dans les résultats.
- **Rendements supposés indépendants** d'un jour à l'autre pour la règle en √h et pour le Monte-Carlo. En pratique les crises sont plus dures que ce que prévoit ce modèle.

- **Volatilité déterministe dans le Monte-Carlo** : la trajectoire de volatilité est celle prévue, elle ne varie pas d'une simulation à l'autre. L'incertitude sur la volatilité elle-même n'est donc pas incluse.
- **Historique court sur la BVMT** : certaines valeurs ont peu de séances et sont peu liquides ; les estimations y sont moins robustes. Les prix ne tiennent pas compte de la liquidité (impossibilité de vendre au prix affiché).
- **Analyse unitaire** : le module traite un actif à la fois. Le risque d'un portefeuille dépend aussi des corrélations (voir partie 6).

Tests

Les 12 tests pytest vérifient, entre autres : que la volatilité estimée retrouve la valeur utilisée pour générer les données ; que ES ≥ VaR et que la VaR à 99 % dépasse celle à 95 % ; le drawdown sur une série connue (100 → 120 → 60 → 90 donne exactement 50 %) ; le rejet des prix invalides ; la mise à l'échelle en √h ; l'estimation GARCH et le retour vers la moyenne ; le repli sur EWMA ; la reproductibilité et la neutralité du Monte-Carlo à dérive nulle ; et l'API (réponse complète, erreurs 422). Ils sont exécutés par la CI à chaque pull request.

6. L'optimisation de portefeuille

6.1 À quoi ça sert

Quand on détient plusieurs actifs, le risque du portefeuille n'est **pas** la moyenne des risques : il dépend de la façon dont les actifs bougent **ensemble** (leur corrélation). Deux actifs qui ne montent et ne baissent pas aux mêmes moments se compensent partiellement : c'est la **diversification**. L'optimiseur de Markowitz cherche la répartition du capital qui exploite cet effet au mieux.

Exemple à la main. Actif A : volatilité 20 %. Actif B : volatilité 10 %. Corrélation 0,3. On cherche les poids qui minimisent le risque :

$$w_A = (\sigma_B^2 - \rho\sigma_A\sigma_B) / (\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B) = (0{,}010 - 0{,}006) / (0{,}040 + 0{,}010 - 0{,}012) \approx 10{,}5\ \%$$

Le portefeuille 10,5 % A + 89,5 % B a une volatilité d'environ **9,8 %**, soit **moins que l'actif B seul (10 %)**, alors qu'on y a ajouté l'actif A, deux fois plus risqué. C'est le gain de diversification. L'optimiseur fait ce calcul pour jusqu'à 15 actifs à la fois, avec des contraintes.

6.2 L'écran « Optimiseur de portefeuille (Markowitz) »

L'écran, dans l'onglet Portefeuille, expose quatre réglages :

Réglage de l'écran	Valeurs	Rôle dans le calcul
Actifs (n/15)	2 à 15 actifs	Les actifs à répartir. Vide : on utilise les positions détenues. Le bouton « Ajouter les positions détenues » les ajoute à la sélection.
Historique	120 à 1 095 jours (365 par défaut)	Fenêtre de données avant la date simulée sur laquelle rendements et risques sont estimés.
Poids maximal par actif	10 % à 100 % (40 % par défaut)	Plafond de concentration : aucun actif ne pèse plus que ce pourcentage.
Taux sans risque (TMM)	0 à 12 % (7 % par défaut)	Rendement d'un placement sans risque ; sert de référence pour le ratio de Sharpe.

Pourquoi le message orange « Avec 2 actifs, il faut au moins 50 % » ? La somme des poids doit faire 100 %. Avec 2 actifs plafonnés à 40 %, le maximum investissable est 2 × 40 % = 80 % : il est impossible d'investir tout le capital. Il faut donc poids maximal × nombre d'actifs ≥ 100 %. Le bouton « Optimiser » est bloqué tant que ce n'est pas respecté ; le service IA refuse aussi la requête (erreur 422) par sécurité.

6.3 Chaîne de calcul

N°	Où	Ce qui se passe
1	Front : PortfolioOptimizerPanel	Envoie la date simulée, les actifs, la fenêtre, le poids maximal et le taux sans risque à /api/v1/ai/optimize.
2	Spring : PortfolioLabService	Charge l'historique de chaque actif, le **coupe à la date simulée**, ne garde que les séances communes à tous les actifs, calcule le nombre de séances par an réel (pour annualiser correctement un mélange bourse / crypto) et calcule les poids actuels à partir de la valeur de marché des positions.
3	FastAPI : portfolio_lab.py	Route /api/ai/optimize : calcule rendements, covariance, trois portefeuilles optimaux, la frontière efficiente et les contributions au risque.
4	Front	Affiche les portefeuilles, la frontière et la comparaison avec les poids actuels.

6.4 Les mathématiques, étape par étape

a) Rendements attendus μ. Moyenne des rendements logarithmiques de chaque actif, multipliée par le nombre de séances par an (252 par défaut ; calculé sur les données réelles quand l'appel vient de Spring).

b) Matrice de covariance Σ. Elle mesure la volatilité de chaque actif et la façon dont les actifs bougent ensemble. Le code utilise l'estimateur de **Ledoit-Wolf** (scikit-learn) plutôt que la covariance empirique : avec peu de séances et plusieurs actifs, la covariance empirique est bruitée et l'optimiseur « se trompe » en exploitant ce bruit. Ledoit-Wolf la rapproche légèrement d'une matrice simple, ce qui la rend plus stable.

c) Risque et ratio de Sharpe d'un portefeuille de poids w.

$$\textit{rendement} = w^T\mu \quad \textit{volatilité} = \sqrt{(w^T\Sigma w)} \quad \textit{Sharpe} = (\textit{rendement} - \textit{taux sans risque}) / \textit{volatilité}$$

d) Les trois portefeuilles calculés (plus le portefeuille actuel du joueur, s'il en a un, pour comparer) :

- **Variance minimale** : minimise $w^T\Sigma w$. Le portefeuille le moins risqué possible.
- **Sharpe maximal** : maximise le Sharpe. Le meilleur rendement par unité de risque. C'est le portefeuille mis en avant par défaut dans l'écran.
- **Poids égaux** : 1/n dans chaque actif. Sert de référence simple : un optimiseur qui ne bat pas le 1/n ne vaut pas sa complexité.

e) Contraintes. Toujours : somme des poids = 1, chaque poids entre 0 et le poids maximal. Le minimum à 0 interdit la **vente à découvert** (« long-only »). Le problème est résolu par l'algorithme **SLSQP** de scipy, qui part d'un portefeuille à poids égaux.

f) Frontière efficiente. L'ensemble des meilleurs portefeuilles possibles : pour chaque rendement cible entre celui du portefeuille de variance minimale et le maximum atteignable, le code cherche la variance la plus faible. Les 5 à 60 points de la courbe (20 par défaut) montrent le compromis : pour gagner plus, il faut accepter plus de risque.

g) Contribution au risque. Pour chaque actif : $w_i \times (\Sigma w)_i / \sigma^2_{\text{portefeuille}}$. Les contributions somment à 100 % et montrent **d'où vient réellement le risque**, qui n'est pas proportionnel au poids : un actif volatil à 25 % de poids peut représenter plus de la moitié du risque.

6.5 Exemple sur quatre actifs simulés

Quatre actifs fictifs (BANQUE, ASSUR, TECH, OBLIG), 400 séances, poids maximal 50 %, taux sans risque 2 % :

Portefeuille	Répartition	Rendement	Volatilité	Sharpe
Variance minimale	BANQUE 4 %, ASSUR 45 %, TECH 1 %, OBLIG 50 %	14,1 %	7,6 %	1,58
Sharpe maximal	BANQUE 39 %, ASSUR 50 %, TECH 1 %, OBLIG 10 %	25,4 %	11,9 %	1,96
Poids égaux (25 % chacun)	BANQUE 25 %, ASSUR 25 %, TECH 25 %, OBLIG 25 %	21,5 %	12,5 %	1,56

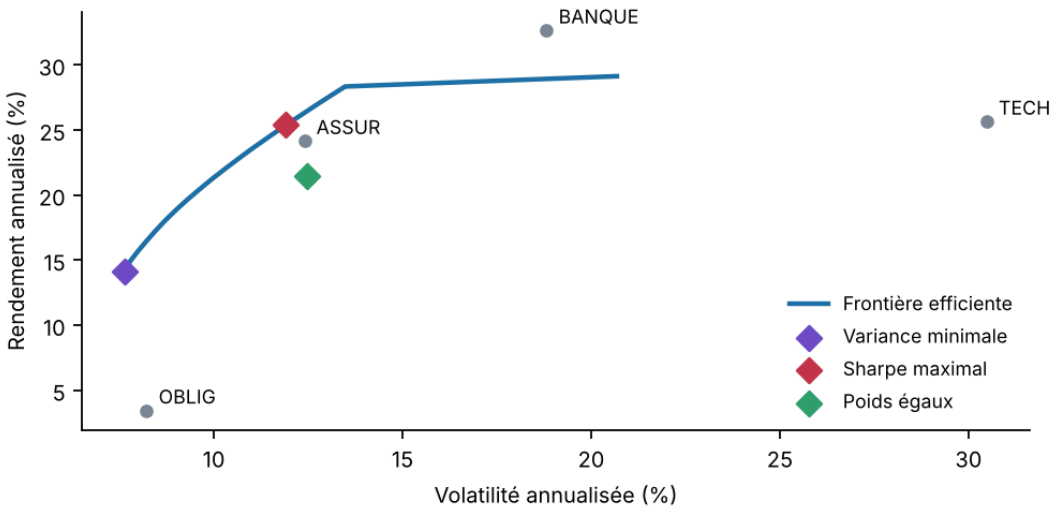


Figure 5 - Frontière efficiente (courbe), actifs seuls (gris) et trois portefeuilles optimaux (losanges).

Lecture. Le portefeuille « Sharpe maximal » offre 25,4 % de rendement pour 11,9 % de volatilité (Sharpe 1,96), contre 21,5 % pour 12,5 % (Sharpe 1,56) avec des poids égaux. Le portefeuille à poids égaux est piégé par TECH : avec 25 % du capital, TECH représente **51 % du risque** (voir la contribution au risque), alors que l'optimiseur le réduit à presque rien. Le portefeuille de variance minimale, lui, descend à 7,6 % de volatilité en misant sur ASSUR et OBLIG, au prix d'un rendement plus faible.

6.6 Limites et bonnes pratiques

- **Les rendements passés sont de mauvais prédicteurs.** L'optimiseur a tendance à « sur-miser » sur les actifs qui ont le mieux performé récemment. C'est pour cela que le projet limite les poids (plafond) et utilise Ledoit-Wolf. L'écran le rappelle : les performances passées ne préjugent pas des performances futures.
- **Aucune donnée future.** Spring coupe l'historique à la date simulée de la salle ; l'optimiseur ne « triche » donc pas en voyant l'avenir.
- **Pas de frais ni de liquidité.** Passer du portefeuille actuel au portefeuille optimal a un coût (commissions, écart achat-vente) qui n'est pas modélisé.
- **Un seul indicateur de risque : la variance.** Markowitz ne distingue pas les hausses des baisses et ignore les queues épaisses. C'est là que l'actuariat complète l'optimiseur.

6.7 Lien entre l'actuariat et l'optimisation

Les deux modules se complètent : l'**optimiseur** propose une répartition, et le module **actuariat** permet de mesurer le risque de chaque actif choisi (VaR, Expected Shortfall, drawdown, fourchette de prix). Une évolution naturelle serait d'appliquer l'analyse actuarielle directement au portefeuille optimisé (VaR du portefeuille, et non plus actif par actif) ; elle n'est pas implémentée à ce jour.

Sources : code du projet PIF (ai/actuarial, ai/portfolio_lab.py, backend com.pif.actuarial et com.pif.ai.insight, frontend ActuarialPage.jsx et PortfolioOptimizerPanel.jsx). Références méthodologiques : Markowitz (1952), Ledoit et Wolf (2004), RiskMetrics (J.P. Morgan, 1996), Bollerslev (1986) pour GARCH.